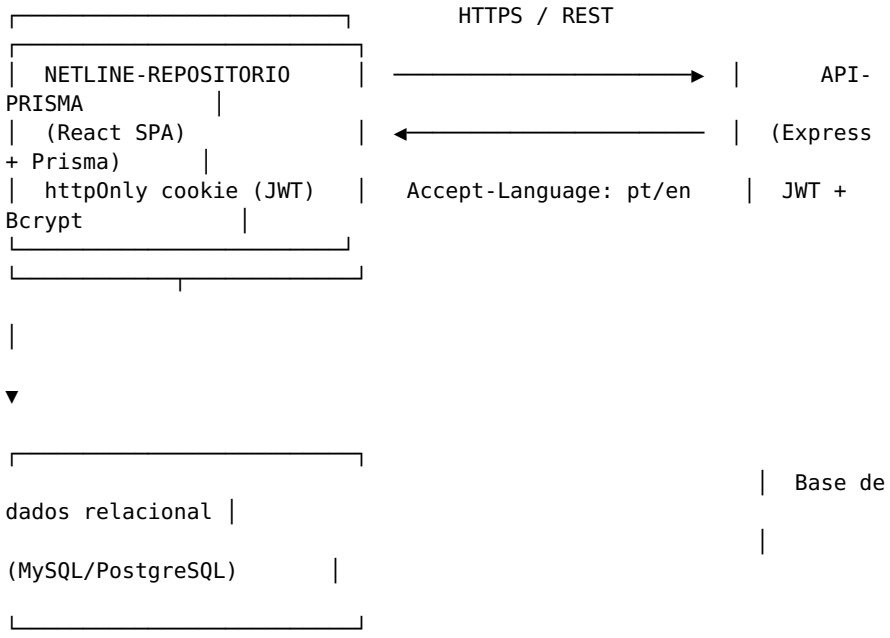
ARQUITETURA

Arquitetura do Sistema de Gestão de Documentos

Duas aplicações separadas que comunicam via API REST:

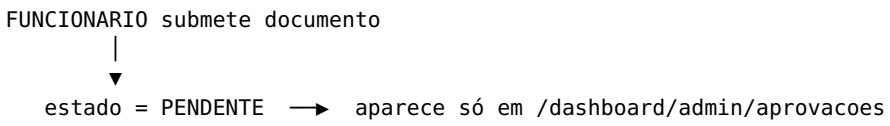
- **API-PRISMA** — backend (Node.js/Express/Prisma)
- **NETLINE-REPOSITORIO** — frontend (Vite/React/TypeScript)

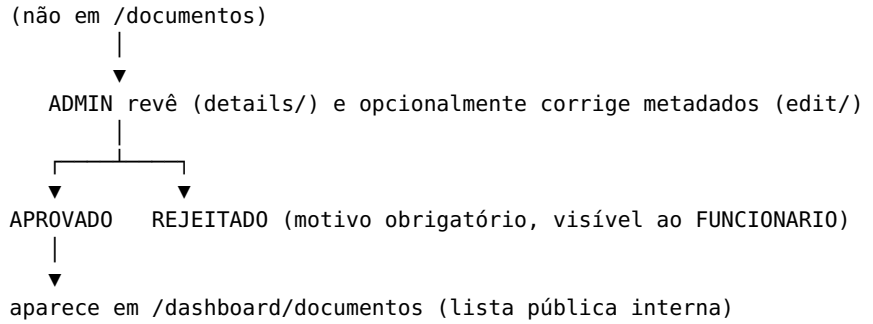


Fluxo de Autenticação

1. Login → backend valida credenciais (Bcrypt) → emite JWT.
2. Token guardado em cookie **httpOnly** (não `localStorage`, por decisão de segurança da empresa).
3. `authMiddleware` valida o token em cada pedido protegido.
4. `roleMiddleware` (`authorize(ROLE)`) restringe endpoints por `perfil`.
5. No frontend, `RotaProtegida` exige sessão válida; `RotaAdmin` exige adicionalmente `perfil === ADMIN` (protege URLs de admin de acesso direto por um FUNCIONARIO).

Fluxo de Documentos e Aprovação





Quando o ADMIN cria um documento diretamente, salta a fila de aprovação — fica `APROVADO` de imediato.

Visibilidade por Sensibilidade de Categoria

- Cada `Categoria` tem `sensivel: boolean`.
- Backend filtra por `perfil` em toda a listagem (categorias e documentos) — FUNCIONARIO nunca recebe nem vê listada uma categoria/documento sensível, mesmo por acesso direto ao id.
- Frontend replica o filtro apenas por UX (ex: dropdown de categoria no formulário de Sistemas) — nunca é a única barreira de segurança.

Soft Delete / Lixeira

- Apagar é soft delete por defeito (`apagadoEm` definido).
- Hard delete exige que o documento já esteja na lixeira (`?definitivo=true`).
- Restauro via `PATCH` com `apagadoEm: null`.
- Tudo exclusivo a ADMIN.

Internacionalização — Sincronização Frontend/Backend

- Frontend guarda o idioma escolhido em `localStorage("idioma")`.
- `http-client.ts` envia esse valor como header `Accept-Language` em cada pedido.
- Backend (`languageMiddleware`) lê o header, define o idioma no `AsyncLocalStorage` do pedido.
- `utils/messages.js` é um Proxy — todo o código que já chamava `MESSAGES.X.Y` passa a devolver a tradução certa automaticamente, sem alterações nos pontos de chamada.

Cache

- Camada de cache in-memory (`node-cache`) aplicada primeiro a `categoriaService.listarCategorias`.
- Chave inclui o `perfil` (`categorias:${perfil}`) — decisão deliberada para nunca servir ao FUNCIONARIO uma resposta cacheada gerada para o ADMIN.
- Redis está planeado para quando o sistema precisar de múltiplas

instâncias (cache partilhada entre processos).

Decisões de Design Relevantes

Decisão	Razão
Auto-registo removido	Só o ADMIN deve poder criar contas — controlo de acesso centralizado
`PATCH /documentos/:id` único para estado, edição e restauro	Endpoints RESTful baseados em recurso, sem verbos no caminho
Sem rota `PUT /auth/me` dedicada	Edição de perfil (mesmo o do ADMIN) reaproveita `PATCH /usuarios/:id`, evitando duplicar lógica
`PATCH /usuarios/me/fotografia` registada antes de `/usuarios/:id`	Evita o Express interpretar `me` como valor de `:id`
Whitelist `CAMPOS_PERMITIDOS_ATUALIZACAO` em `atualizarUsuario`	Previne mass assignment a partir de `req.body`
Categorias seedadas, não criadas via API	Evita categorias quase-duplicadas e desorganização
Página vs. componente: subpasta local só se usada numa única página	Mantém `components/` como reutilização genuína, não um "cesto" geral
Preview de documento em rota própria, não modal	Melhor comportamento em mobile/tablet; viewer nativo de PDF não era suficiente

Pendências Transversais

- Log de atividade real (backend) para alimentar o dashboard e permitir tradução por código de ação em vez de texto pré-formatado.
- Uniformizar `Usuarios` e `Lixeira` no padrão `pages/admin/` do frontend.
- Esconder no cliente o botão "Apagar" para não-admins (o backend já bloqueia, falta o reflexo na UI).